Software Design Document

for

Compile And Conquer

Team: Compile And Conquer

Project: Compile And Conquer

Team Members:
Jonah Jellison
Noah Schmalenberger
Joshua Oliver
Nicole Fuller
Emily Porter

Last Updated: 3/23/2025

Table of Contents

Table of Contents..... 1
Document Revision History..... 3
List of Figures..... 4
1. Introduction..... 5
    1.1 Architectural Design Goals..... 5
2. Software Architecture..... 6
    2.1 Overview..... 8
    2.2 Subsystem Decomposition..... 8
3. Subsystem Services..... 12

Document Revision History

Revision Number	Revision Date	Description	Rationale
1.0	3/23/2025	Submission Draft	
2.0	4/28/25	Milestone 3 Submission Draft	

List of Figures

Figure #	Title	Page #
1	Class Diagram	6
2	Component Diagram	10

1. Introduction

Our application is a bullet hell game where the player is being attacked by four different enemies, each with their own types of attacks. Only one enemy is attacking the player at a time, with each enemy having increasing health from the last. The player's goal is to defeat each of the enemies while dodging their bullets. The player has three lives to do so and once they run out of lives, it's game over.

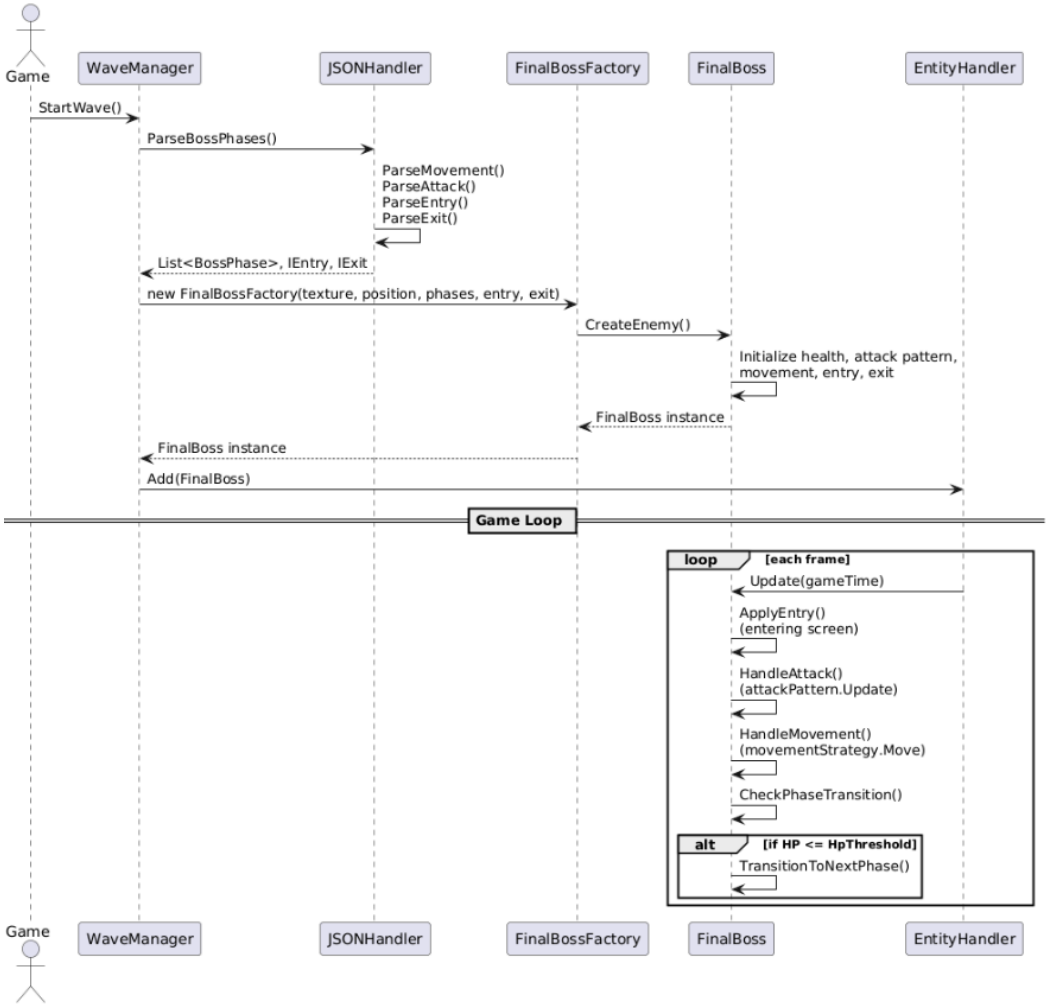
1.1 Architectural Design Goals

One of the design qualities that applies to our system is performance. Since we are creating a game application, performance is an important aspect of the system. The two tactics for the performance quality are control resource demand and manage resources. Our application follows the control resource demand tactic as it manages sampling rate, limits event response, and prioritizes events. In the application, events are used to follow the player life system. When the player loses a life, the event gets triggered which sends a signal to the user interface screen so that one of the stars representing the player's lives gets removed. The same logic will be used when the scoring system is implemented. Through the Singleton design pattern, we can also implement lazy instantiation to reduce startup time. At the moment this is not fully implemented, but we aim to implement it to improve application performance.

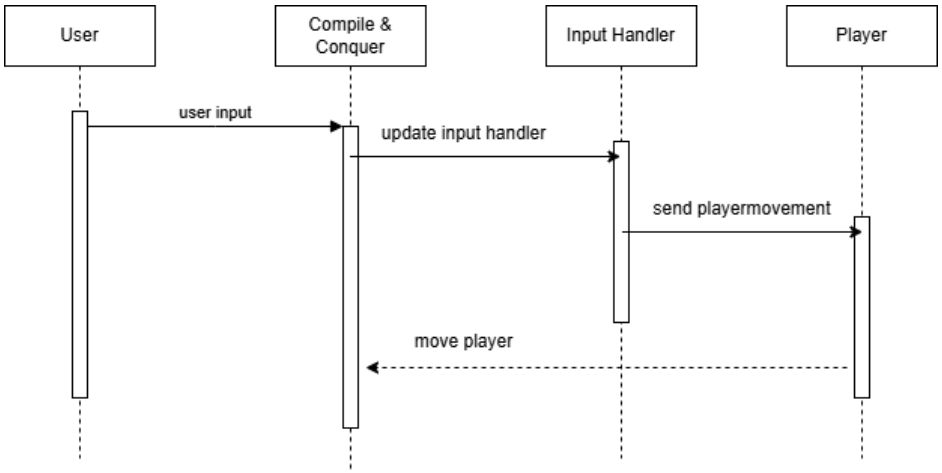
The second design quality that we chose was availability. As we look forward to milestone 3, we anticipate issues in increasing dynamicity for our system. We will employ tactics like detecting faults to check when exceptions are thrown. Incorporating exception detection will allow us to detect when an error occurs and make sure our code stays running or exits safely. This connects to the tactic of recovering from faults. If our code does throw an exception, we will use exception handling to ensure the availability of the software. For instance, if there is an error when reading in the JSON file, we can input default values instead or allow the user to restart the program. Overall, as we progress through the semester, we can implement availability by designing with availability in mind. By anticipating what faults may occur, we can add exception prevention. For the singleton patterns, we could input thread synchronization to prevent multiple instantiations. By identifying systems that will most likely produce more faults than others, we can add more exception detection, exception handling, and exception prevention in those areas.

2. Software Architecture

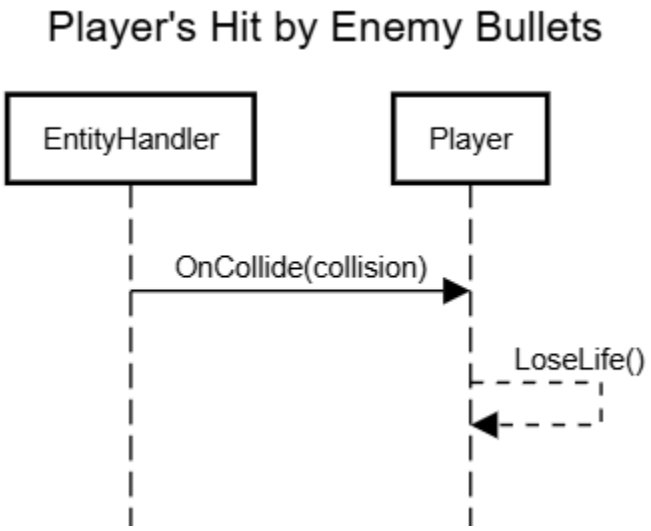
Use case 1: WaveManager spawns FinalBoss



Use case 2: User controls player to move



Use case 3: Player is hit by enemy bullets



2.1 Overview

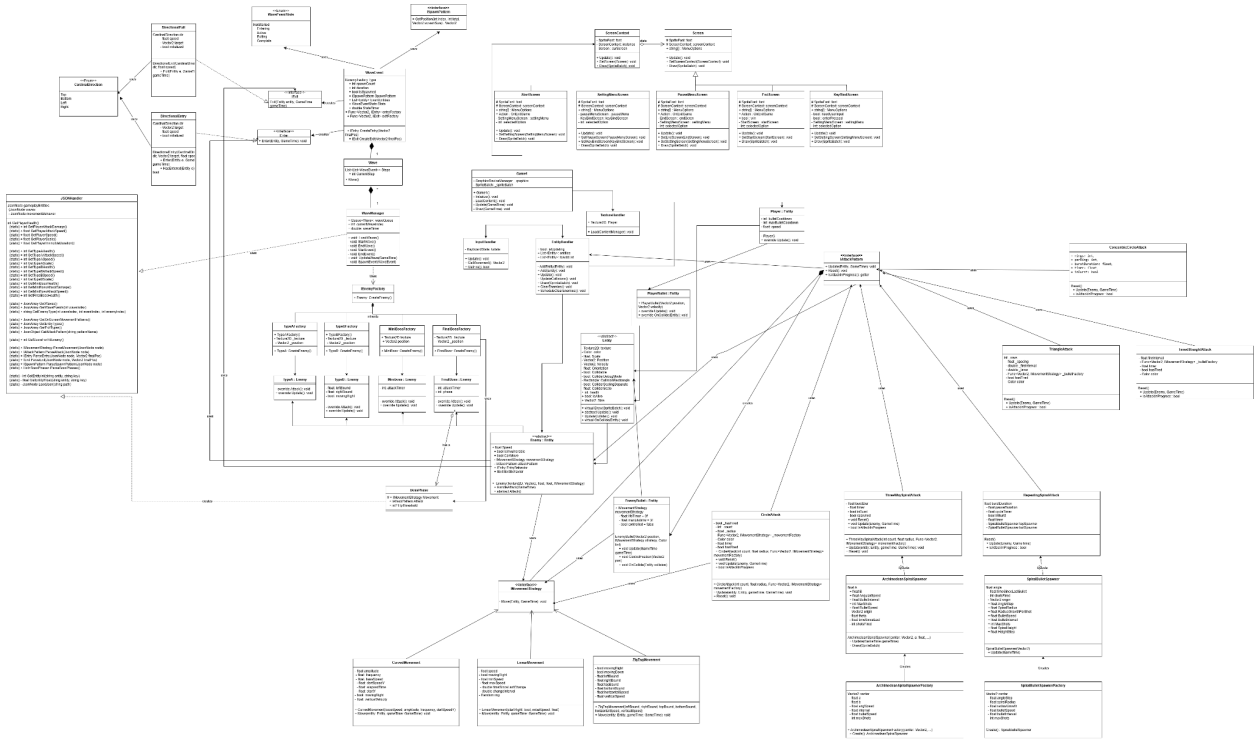


Figure 1

Our system follows an MVC architecture, dividing responsibilities between distinct layers. The model layer encompasses the data and logic of the game with entities like player, enemy, various movement types, attack patterns, and behaviors. The view layer is managed by classes like screen which defines the visual components of the game states. The controller layer manages user input and game flow with classes like inputHandler and WaveManager.

2.2 Subsystem Decomposition

This section describes how the game system is broken down into modular subsystems. Each subsystem encapsulates specific responsibilities to support maintainability, extensibility, and overall gameplay logic. These subsystems allow us to follow the model-view-controller architecture pattern, promoting good cohesion and coupling practices.

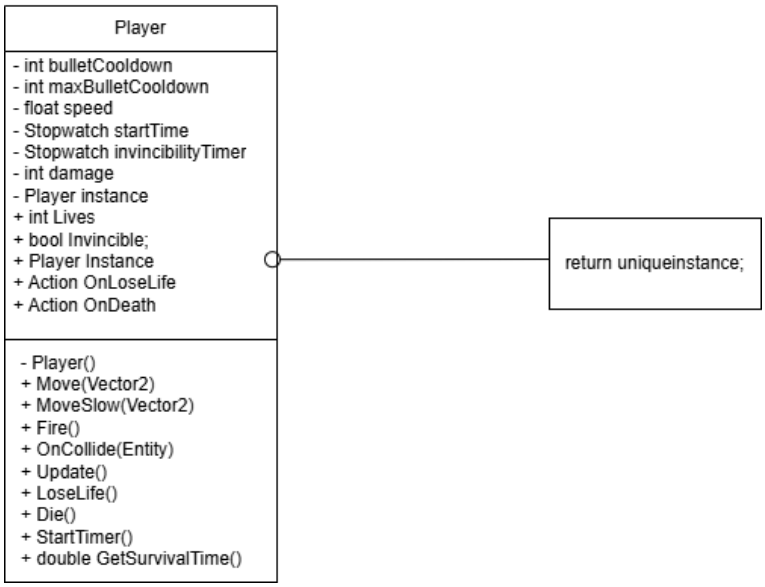
2.2.1 Screens

The screens subsystem currently contains the three screens other than the gameplay screen. The screens within this subsystem include the start menu, game over screen, setting screen, keybind screen, and pause menu. The screens allow the player to make a selection based on the offered menu options.

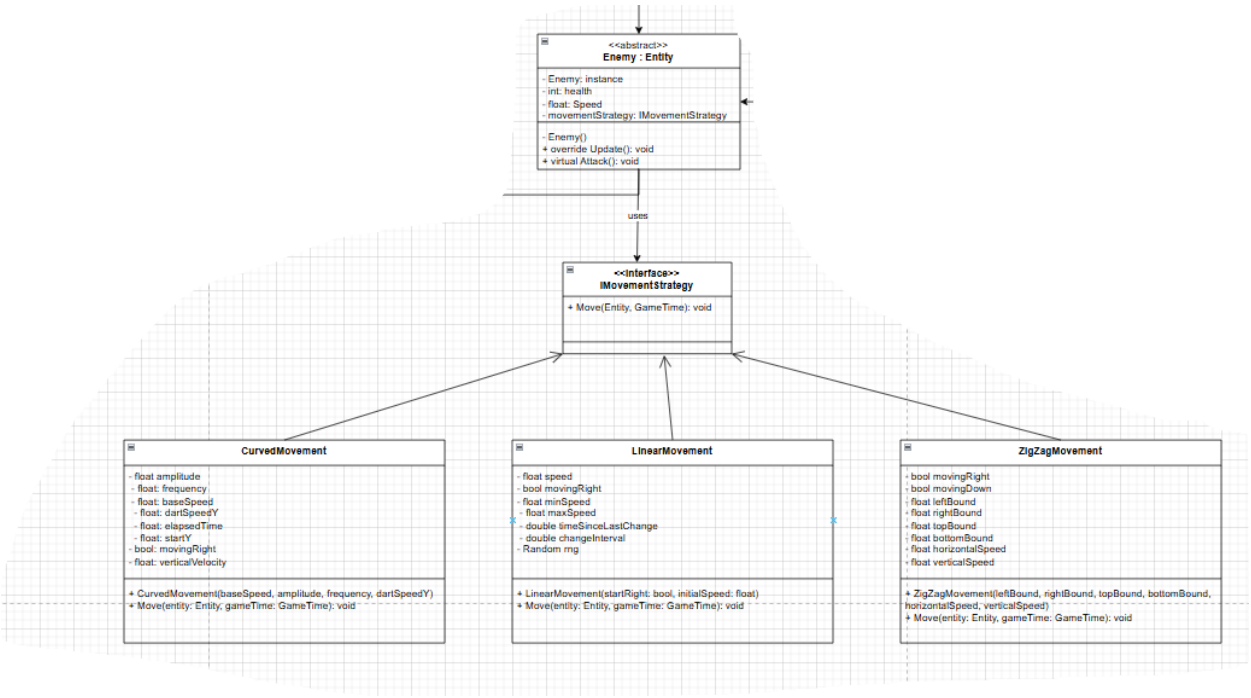
2.2.2 Design Patterns

Singleton (Player)

The singleton design pattern is very easy to implement into code. We chose to use this design pattern for the player since there only should ever be one player in the game. We also used this pattern for the different screens within the game.

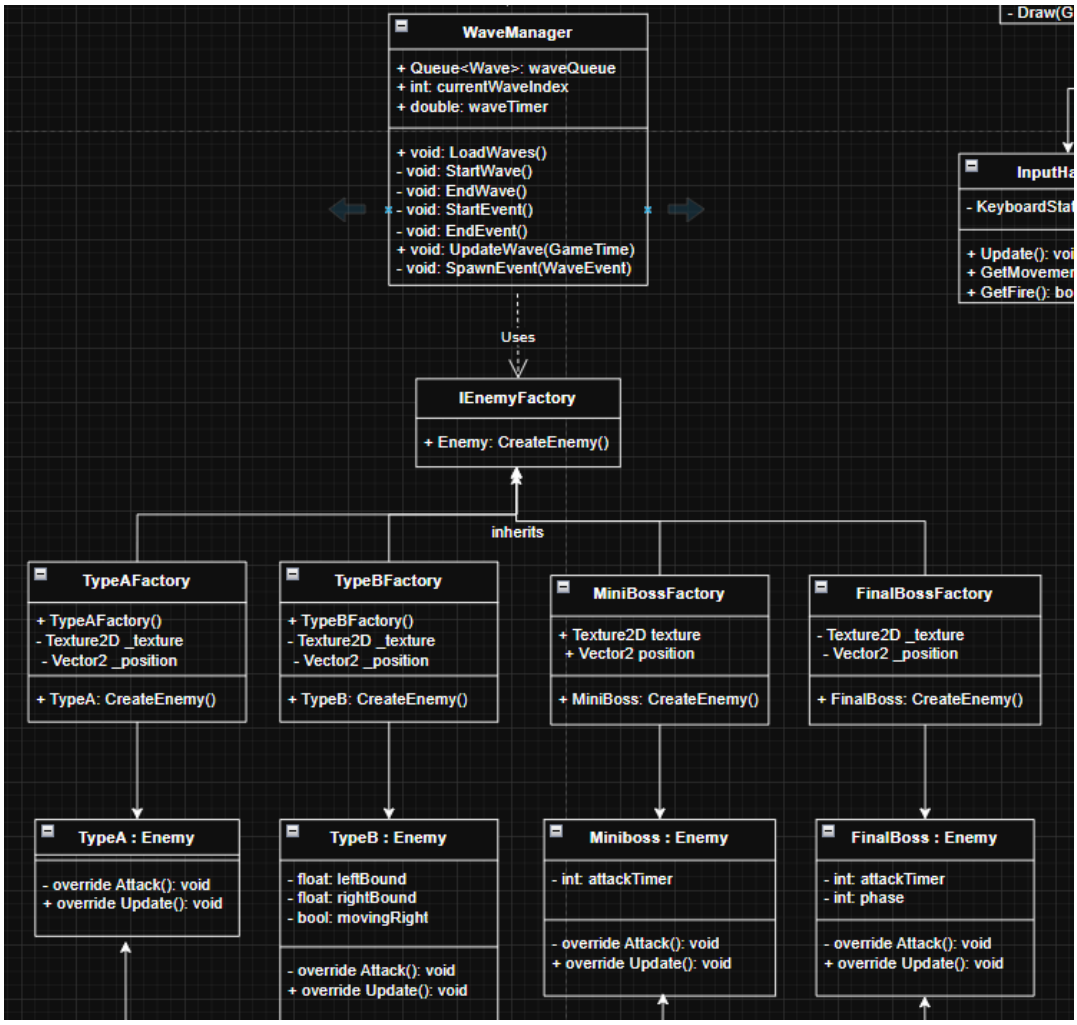


Command Pattern (Movement)



We use the Command Pattern to encapsulate different enemy movement types as standalone command objects. We use the IMovementStrategy interface and its concrete implementations, LinearMovement, CurvedMovement, and ZigZagMovement to do so. Each enemy gets a movement command at instantiation and executes it in their Update() function.

Abstract Factory Pattern (Enemy Creation)



The abstract factory pattern has been implemented to facilitate the creation of enemies. The wave manager uses a pseudo-builder pattern to create waves of enemies in incremental events. When an event is created, a copy of the specified type factory is stored with parameters such as the entry, exit, and movement pattern to be used with the entity. The factory pattern is very useful for modularising the creation of specific entities for creation at any point during the wave.

3. Subsystem Services

Component Diagram:

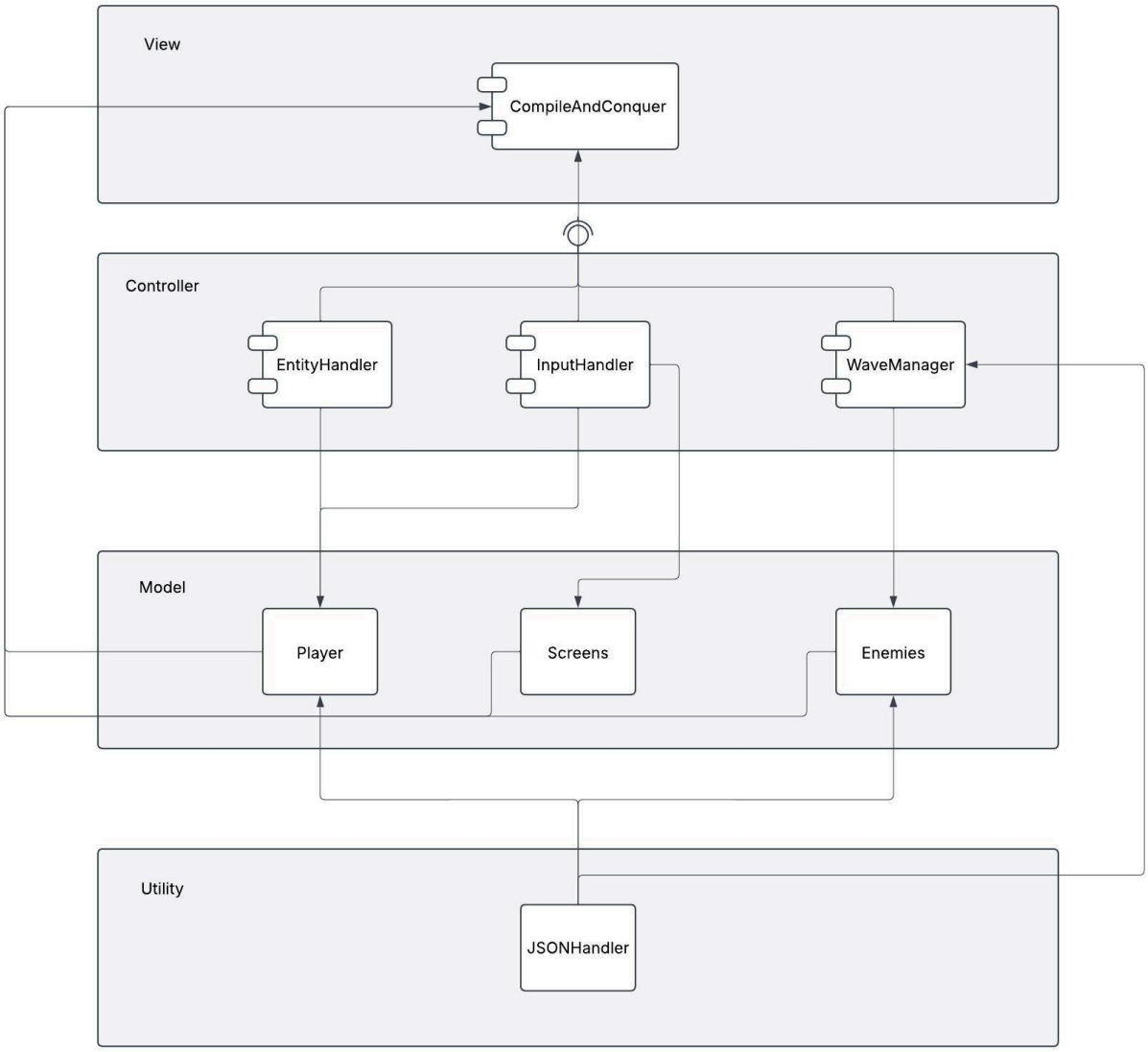


Figure 2

CompileAndConquer: This component handles our main game loop and also draws all models to the screen. To do this it gets the information it needs to draw each entity or screen from our models. It also initializes our controllers which then handle the inputs to our view.

EntityHandler: This subsystem is in charge of all entities. It updates each entity, checks collisions, and also removes them when they are dead. It then affects all entities by allowing them to change their state.

InputHandler: This subsystem handles all input our view experiences. It then passes that information along to the screens and player, allowing them to use that information to move or adjust what is shown.

WaveManager: This subsystem controls our enemies by parsing enemy wave data from JSON using JSONHandler. It instantiates enemy objects using Abstract Factory Pattern, like TypeAFactory, TypeBFactory, etc. It assigns enemies an Entry and Exit pattern (IEntry, IExit), and controls the timings for spawning enemies, transitioning between wave steps, and ending waves. Also manages the progression of boss phases for FinalBoss encounters.

Player/Screens/Enemies: These are our models which store the game state and information about everything displayed to the screen. They are each controlled by their respective handler, and their state is what allows our CompileAndConquer component to draw everything to the screen.

JSONHandler: Parses configuration files for gameplay entities, movement behaviors, wave events, attack patterns, entry/exit behaviors, and spawn patterns. It provides data used by WaveManager to construct enemies dynamically. Also gives default gameplay settings for player health, enemy stats, and attack behaviors.